BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 3

Roll no: 08 PRN No: 12412097 Name of Student: Sahil Balasaheb Patil

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

1) Start the program.

2) Import the required libraries:

- TensorFlow/Keras
- NumPy
- Matplotlib
- Scikit-learn

3) Load the Iris dataset using Scikit-learn.

4) Display the dataset information:

- Shape of the dataset
- Number of input features
- Number of target classes

5) Preprocess the dataset:

- Separate the input features (**X**) and target labels (**y**).
- Split the dataset into training and testing sets using an **80:20 ratio**.
- Normalize the input features using **StandardScaler**.

6) Define a function to create a Multilayer Perceptron (MLP):

- Input layer with **4 neurons** (Iris features).
- Hidden layer with **10 neurons (ReLU activation)**.

- Hidden layer with **8 neurons (ReLU activation)**.
- Output layer with **3 neurons (Softmax activation)**.
- Use **Stochastic Gradient Descent (SGD)** optimizer with different learning rates.

7) Compile the model using:

- **Loss Function:** Sparse Categorical Crossentropy
- **Optimizer:** SGD
- **Evaluation Metric:** Accuracy

8) Train the model using:

- Different **learning rates** (0.001, 0.01, 0.1)
- Different **epochs** (20, 50, 100)
- Validation split of **20%**
- During training:
    - **Forward Propagation:** Computes predictions by passing inputs through all layers.
    - **Backpropagation:** Computes gradients of the loss and updates weights using SGD.

9) Evaluate the trained model on the testing dataset and record the test accuracy for each combination of learning rate and epochs.

10) Plot the training and validation accuracy curves for one selected model (Learning Rate = **0.01**, Epochs = **50**).

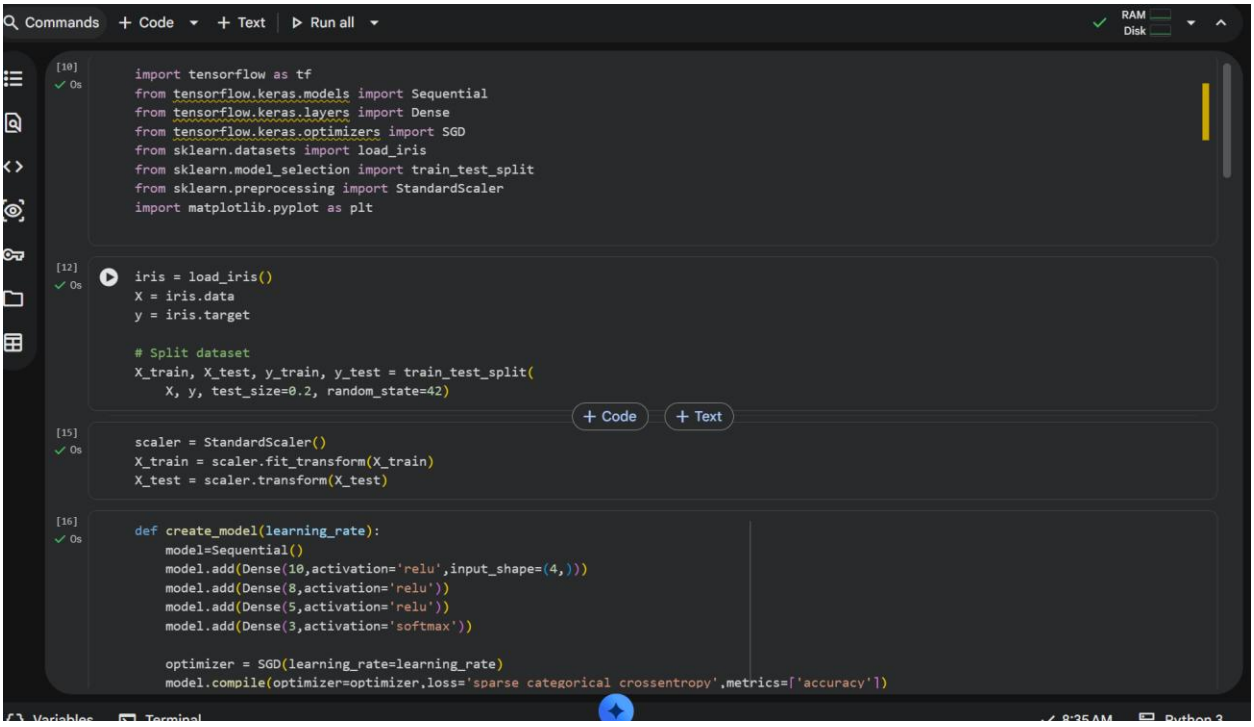
11) Display a summary table showing:

- Learning Rate
- Number of Epochs
- Test Accuracy

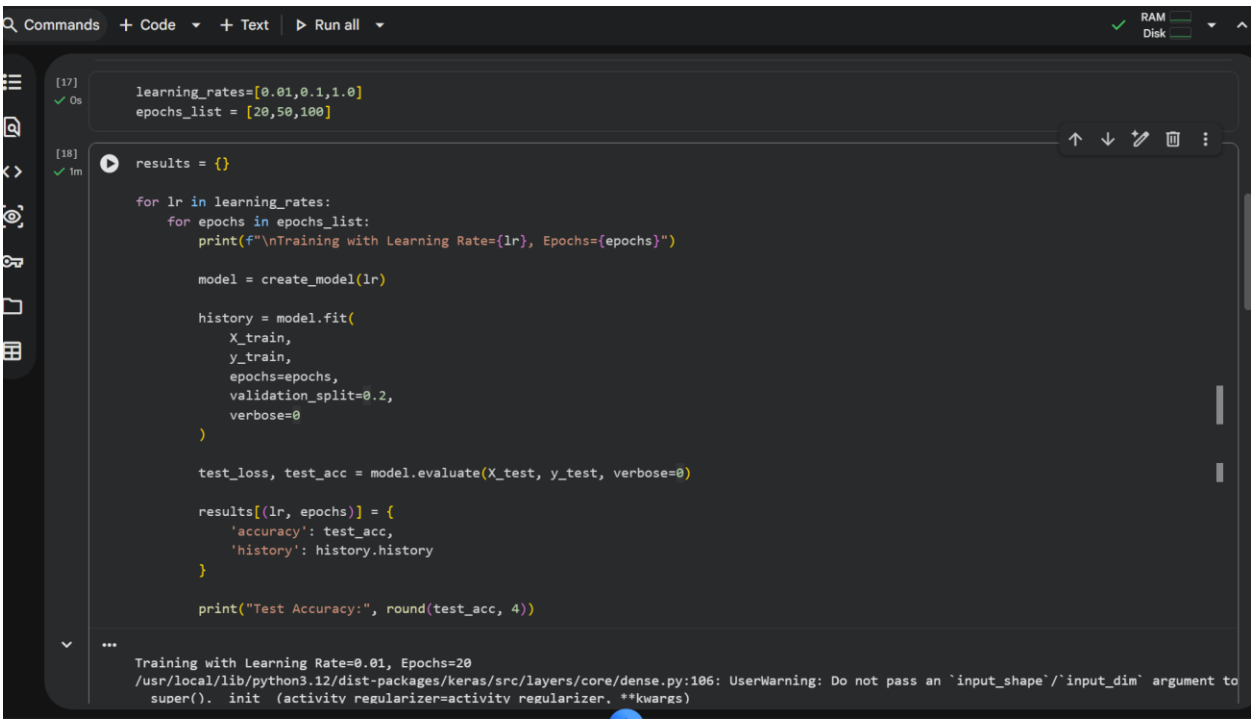
12) Compare the results and analyze how learning rate and epochs affect the model performance.

13) Stop the program.

Output :



```
[10] ✓ 0s
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

[12] ✓ 0s
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

[15] ✓ 0s
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

[16] ✓ 0s
def create_model(learning_rate):
    model=Sequential()
    model.add(Dense(10,activation='relu',input_shape=(4,)))
    model.add(Dense(8,activation='relu'))
    model.add(Dense(5,activation='relu'))
    model.add(Dense(3,activation='softmax'))

    optimizer = SGD(learning_rate=learning_rate)
    model.compile(optimizer=optimizer,loss='sparse categorical crossentropy',metrics=['accuracy'])
```



```
[17] ✓ 0s
learning_rates=[0.01,0.1,1.0]
epochs_list = [20,50,100]

[18] ✓ 1m
results = {}

for lr in learning_rates:
    for epochs in epochs_list:
        print(f"\nTraining with Learning Rate={lr}, Epochs={epochs}")

        model = create_model(lr)

        history = model.fit(
            X_train,
            y_train,
            epochs=epochs,
            validation_split=0.2,
            verbose=0
        )

        test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

        results[(lr, epochs)] = {
            'accuracy': test_acc,
            'history': history.history
        }

        print("Test Accuracy:", round(test_acc, 4))

Training with Learning Rate=0.01, Epochs=20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Commands + Code + Text ▷ Run all RAM Disk

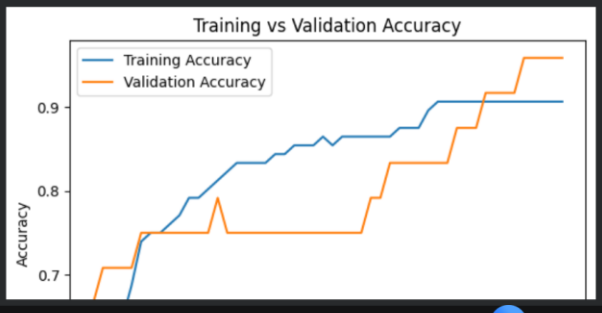
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/ to
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Test Accuracy: 0.1

Training with Learning Rate=0.01, Epochs=50
Test Accuracy: 0.9333

Training with Learning Rate=0.01, Epochs=100
Test Accuracy: 0.6333

Training with Learning Rate=0.1, Epochs=20
Test Accuracy: 0.9333

Training with Learning Rate=0.1, Epochs=50
Test Accuracy: 1.0

Training with Learning Rate=0.1, Epochs=100
Test Accuracy: 0.9667

Training with Learning Rate=1.0, Epochs=20
Test Accuracy: 1.0

Training with Learning Rate=1.0, Epochs=50
Test Accuracy: 0.9667

Training with Learning Rate=1.0, Epochs=100
Test Accuracy: 0.9667
```

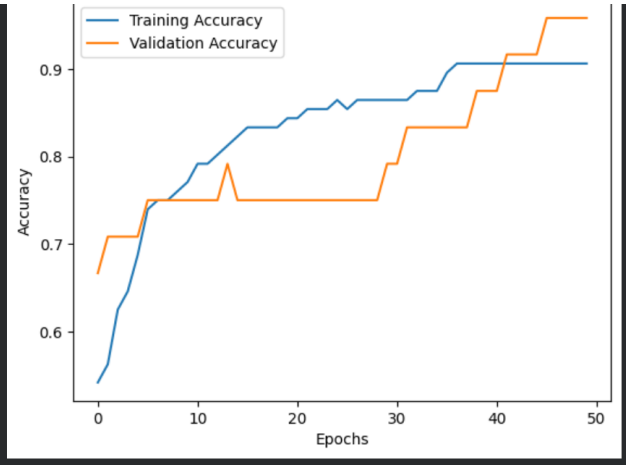
[19] ✓ 0s

```
history = results[(0.01, 50)]['history']

plt.plot(history['accuracy'], label='Training Accuracy')
plt.plot(history['val_accuracy'], label='Validation Accuracy')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
```

[19] ✓ 0s

```
history = results[(0.01, 50)]['history']

plt.plot(history['accuracy'], label='Training Accuracy')
plt.plot(history['val_accuracy'], label='Validation Accuracy')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()

print("\nSummary")
print("-----------------------------------------")
print("Learning Rate\tEpochs\tTest Accuracy")
print("-----------------------------------------")

for key, value in results.items():
    print(f"{key[0]}\t\t{key[1]}\t{value['accuracy']:.4f}")
```






```
Summary
------------------------------------------
Learning Rate   Epochs  Test Accuracy
------------------------------------------
0.01        20  0.1000
0.01        50  0.9333
0.01        100 0.6333
0.1     20  0.9333
0.1     50  1.0000
0.1     100 0.9667
1.0     20  1.0000
```